SOKOLINK

SokoLink — Ways of Working

Version 2 · 2026-08-05. Replaces the TypeScript-era v1. **Companion docs:** `docs/PRODUCTION_PLAN.md` says *what* we build and in what order. This says *how* we work.

---

1. Roles

GURU	CLAUDE
Product decisions, final say on scope	Writes the code, proposes the approach
Runs and tests on a real phone	Keeps Notion in sync with reality
Railway deploys, secrets, Meta/Daraja credentials	Flags risk, scope creep and slippage early
Recruits the real sellers	Prepares commits and PR descriptions
Owns the Notion workspace and the repo	Never touches secrets; never pushes without a go-ahead

---

2. The working loop

Every session runs in three beats.

Open. Claude pulls the current milestone's task from Notion and restates the scope in one short message. Guru confirms or cuts. *Nothing starts before we agree what "now" is.*

Build. Claude works one task at a time and hands over something runnable. Guru tests on a real phone — not just localhost.

Close. Claude updates Notion, writes a short log of what shipped and what's blocked, and stages the commit.

> **Pace is not fixed.** The milestone numbering is a **dependency order, not a schedule**. If three tasks can land in one sitting, they land — we just close each Notion row as it does.

---

3. Definition of done

A task is **Done** only when all six hold:

1. Code merged to `main`
2. Runs on a real device or on Railway
3. Tests pass, CI green
4. The code documents itself to the standard in §4
5. The Notion row is updated
6. A commit references the task

"It imports" is not done. "It works on localhost" is not done.

---

4. Documentation standard — the one that matters most

> *"Every implementation documented — it makes it easier to understand the codebase, debug, and know why each stage was required."*

This is not decoration. In six weeks nobody remembers why the price cascade escalates in that order, or why the Daraja callback is idempotent. The code has to say so itself.

Every file opens with a header docstring covering three things:

```
"""
Draft agent — a cover image becomes a product draft.

    cover image + caption ──> Gemini (schema=ProductDraft) ──> {name, price?, tags}
                              constrained decoding             seller CONFIRMS

WHY this exists: captions carry no prices (spike 00 proved it against 24 real
captions). The product must be read from the image, and failing that, heard in
the video. This file is the ONLY place that knows which vision provider we use —
swapping models is a change here and nowhere else.
"""
```

1. **What it does**, in one line.
2. **The pipeline**, as an ASCII diagram when data flows through it.
3. **Why it exists and why it is shaped this way** — the decision, the constraint, the thing that would otherwise be re-litigated.

Beyond the header:

- **Docstrings on every public function, class and route** — what it takes, returns, and raises.
- **Comments explain *why*, not *what*.** If code needs a comment to say *what* it does, rewrite the code instead.
- **Non-obvious business rules always get a comment** — Kenyan pricing conventions, Sheng parsing quirks, M-Pesa formats, TikTok CDN expiry. Future us will not remember.
- **Every milestone updates `docs/`** — the plan's status log, and `CONCEPTS.md` when a new *idea* enters the system.

---

5. Engineering standard

Production-grade, ready for real sellers. Not a demo that happens to run.

Type safety and validation

- **Pydantic at every boundary**: API request/response, external API payloads, and — most importantly — **LLM output**. An LLM's JSON shape is never trusted; the schema is the guarantee.
- Type hints throughout. `mypy` clean.
- Settings via `pydantic-settings`. **Never read `os.environ` directly** outside config.

Testing

LAYER	COVERS
Unit	Parsers, price/currency logic, phone normalisation, pure helpers
Integration	API routes and DB queries against real Postgres, in rolled-back transactions
Money paths	Every order and payment state transition, including replays
AI accuracy	A fixture set of real Sheng/Swahili captions and covers, guarding prompt changes

- **Tests ship in the same PR as the logic.** Never "tests later."
- **External services are always mocked.** Tests never hit Apify, Gemini, Meta or Daraja — those cost money and rate-limit.
- Money paths and webhook idempotency get tests that assert *replays change nothing*.

Error handling

- **No silent `except`.** Handle meaningfully or re-raise with context.
- User-facing failures get a real message and a recovery path — never a blank screen or raw JSON.
- Structured logging with a correlation id per conversation and per order, so any incident can be reconstructed.

Security

- Secrets in `.env` (gitignored) and Railway variables. **Never in `.env.example`** — that file is committed, and this has already caught us once.
- **Never put a phone number or identity in a URL.** Links leak via history, referrers, and forwarding. Storefront links carry a signed, scoped, short-lived token.
- Rate limits and input validation on every public route.

CI

GitHub Actions on every PR: `mypy`, `ruff`, and the full test suite. **A red build does not merge.**

---

6. Architectural rules — do not silently change

These were proven in Project TIKTOK and carry forward.

- **The agent proposes, code disposes.** The LLM never decides price, stock, payment status or consent. It drafts; deterministic code transacts.
- **Publish is a human gate**, and still requires a price.
- **The payment callback is the only payment truth**, processed idempotently — Daraja retries, and so does Meta.
- **Rails before agent.** Payment paths are built and tested before an agent may call them.
- **No RAG** — every lookup here is by known key. Revisit only if a genuine similarity problem appears.
- **No AI framework.** Provider SDKs called directly, Pydantic as the guardrail. Nothing magic, so nothing is un-debuggable.
- **One database per application.** Learned the hard way; nearly re-learned worse.

- **Spike before schema.** Let a real payload drive the data model. Spike 00 killed the "captions carry prices" assumption before it reached the DB — that is the pattern.
- **Money is stored as integer KES.** No floats anywhere near a price.
- **Cache anything that costs money.** Apify once per day per seller; each video processed once ever, keyed by video id.

---

7. Git flow

- **One branch per milestone**: `p0-foundation`, `p1-catalog`, `p2-wa-channel`, …
- **PR at the milestone's end.** Claude writes the description; Guru merges.
- Commit messages reference the Notion task they close.
- **Remote is HTTPS** — no SSH keys on the dev machine.

---

8. Source of truth

QUESTION	ANSWER LIVES IN
What's the status of a task?	**Notion**
What does the code actually do?	**The repo**
Why did we choose X over Y?	**Notion** → **Decisions**, and the file header
What are we deliberately not doing?	**Notion** → **Backlog**

Neither system guesses about the other. If they disagree, fix the mismatch before continuing.

---

9. Guardrails

- **SokoLedger stays parked.** New ideas go to Backlog, not into the current milestone.
- **Claude flags, Guru cuts.** When scope won't fit, it gets said out loud. Deciding what to drop is Guru's call.
- **Commerce before Intel before polish.** Nothing matters until a seller can sell.
- **Cost rails are not optional.** Any feature a user can trigger repeatedly needs a cache, a quota, and — where appropriate — a paywall, designed in from the start rather than added after a bill.

---

10. Amending this document

A working agreement, not a contract. If something here is slowing us down, say so and we change it — deliberately, in this file, rather than drifting away from it silently.
